

Ogni elemento viene inserito e rimosso dalla coda al massimo una volta; ogni rimozione ha costo $O(\log k)$. Il costo computazionale di questa soluzione è quindi $O(n \log k)$.

Esercizio A3 – Punti ≥ 12

Questo problema è molto simile al Problema 2 del 24/04/2014. Come in quell'esercizio, è possibile risolverlo con una semplice post-visita dell'albero, di costo $O(n)$. In quell'esercizio trovate anche varianti della soluzione che possono essere adattate a questo problema.

```
(int, boolean) wbRic(TREE t)
```

```

if t == nil then
    return (0, true)
nL, balancedL = wbRic(t.left)
nR, balancedR = wbRic(t.right)
return (nL + nR + 1, balancedL and balancedR and |nL - nR| ≤ 1)
```

```
boolean wellBalanced(TREE T)
```

```

n, balanced = wbRic(T)
return balanced
```

Nella funzione ricorsiva viene restituita una coppia di valori, dove il primo valore è un intero che rappresenta il numero di nodi compresi nel sottoalbero, mentre il secondo è un booleano uguale a **true** se il sottoalbero è ben-bilanciato.

Esercizio B1 – Punto d'incontro – Punti ≥ 8

Il problema è simile all'Esercizio 1 del 14/07/15 - quello di Alice e Bob che si devono incontrare, solo che non c'è Carl e la metrica per scegliere un nodo è leggermente diversa. Notate che la discussione sul minimo è complicata dal fatto che non volevo scrivere direttamente "cammini minimi", quindi ci ho girato un po' intorno.

È sufficiente utilizzare due ricerche dei cammini minimi, a partire dai nodi s e t .

Una volta ottenuti i pesi dei cammini minimi, è sufficiente cercare il nodo in cui pesi sono uguali. La procedura `shortestPath()` implementa l'algoritmo di Johnson con complessità $O(m \log n)$ e restituisce il vettore delle distanze (oltre che all'albero dei cammini minimi, che viene ignorato). Ora, nella mia testa i pesi erano positivi, ma ho scritto che sono in $\mathbb{R}$, quindi qui bisognerebbe applicare Bellman-Ford con costo $O(mn)$ (thanks Loris Lorenzini).

```
int meetingPoint(GRAPH G, int[][] w, NODE s, NODE t)
```

```

int[], int[] ds, Ts = shortestPath(G, s)
int[], int[] dt, Tt = shortestPath(G, t)
foreach u ∈ G.V() do
    if ds[u] == dt[u] and ds[u] ≠ ∞ then
        return true
return false
```

La complessità è dominata da $O(m \log n)$, in quanto la ricerca del nodo con peso uguale ha costo lineare in n .

Esercizio B2 – Punti ≥ 10

Il problema può essere risolto con un approccio basato su backtrack, ovviamente. È sufficiente adattare l'algoritmo che genera tutti i sottoinsiemi di un elemento, con l'accorgimento di non selezionare un valore 1 se il valore precedente è un 1.

```
printBitsRec(int[] S, int n, int i)
```

```
if i == n + 1 then
    print S
else
    if i == 1 or S[i - 1] != 1 then
        S[i] = 1
        printBitsRec(S, n, i + 1)
    S[i] = 0
    printBitsRec(S, n, i + 1)
```

```
printBits(int n)
```

```
int[] S = new int[1...n]
printBitsRec(S, n, 1)
```

È sufficiente dire che la complessità è $O(n \cdot 2^n)$, perché ovviamente le stringhe di n bit sono 2^n , ma non le stamperemo tutte. Più precisamente, abbiamo visto che il numero di stringhe di n bit senza due bit consecutivi cresce come l' $n+2$ -esimo numero di fibonacci, quindi la complessità è $O(nf(n))$ (si veda l'esercizio del 7 Febbraio 2019).

Esercizio B3 – Punti ≥ 12

L'esercizio è simile all'Esercizio 2 del 17/12/2015, solo che invece di chiedere di non poter "saltare" più di k elementi, in quel problema non si potevano "prendere" più di k elementi consecutivi. Basta quindi adattare le formule per risolvere il problema.

Sia $DP[i][j]$ il valore che posso ottenere dai primi i elementi, avendo ancora la possibilità di "cancellare" j elementi consecutivi. In altre parole, per via di selezioni precedenti, posso continuare a cancellare fino a j elementi, ma poi dovrò includere un elemento. La soluzione del problema originale si trova in $DP[n][k]$.

Una formulazione ricorsiva per DP è la seguente:

$$DP[i][j] = \begin{cases} 0 & i = 0 \\ DP[i-1][k] + V[i] & i > 0 \wedge j = 0 \\ \max\{DP[i-1][j-1], DP[i-1][k] + V[i]\} & i > 0 \wedge j > 0 \end{cases}$$

In pratica, se $j = 0$ si è costretti a prendere l'elemento, mentre se $j > 0$ si può scegliere di saltare o non saltare un elemento; se si decide di non saltare, il numero di elementi consecutivi selezionabili si resetta a k e bisogna sommare il valore selezionato; se si decide di saltare, restano a disposizione $j - 1$ elementi consecutivi da cancellare.

È possibile risolvere il problema con memoization nel modo seguente:

```
int maxSumK(int[] V, int n, int k)
```

```
int[][] DP = new int[1...n][1...k] = {-1} % Initialized to -1
return maxSumKRec(V, n, k, DP)
```

```
int maxSumKRec(int[] V, int i, int j, int[][] DP)
```

```
if i <= 0 then
    return 0
if DP[i][j] < 0 then
    if j == 0 then
        DP[i][j] = maxSumKRec(V, i - 1, k, DP) + V[i]
    else
        DP[i][j] = max(maxSumKRec(V, i - 1, j - 1, DP), maxSumKRec(V, i - 1, k, DP) + V[i])
return DP[i][j]
```

La complessità è pari a $O(nk)$, in quanto è necessario riempire tutta la tabella.

Una soluzione alternativa, abbozzata da alcuni studenti, è la seguente. Sia $DP[i]$ il valore della sottosequenza k -contigua massimale per i primi i elementi che termina nella posizione i -esima, cioè che include $V[i]$. Una formulazione ricorsiva per DP è la seguente:

$$DP[i] = \begin{cases} 0 & i \leq 0 \\ \max_{i-k \leq j < i} \{DP[j]\} + V[i] & \end{cases}$$

L'idea è questa: se prendo l'elemento i -esimo, posso saltare 0 elementi (e quindi prendere anche $i - 1$); saltare un elemento ($i - 1$) e prendere $i - 2$; saltare due elementi ($i - 1, i - 2$ e prendere $i - 3$); fino a quando non raggiungo $i - k - 1$. Oltre non posso andare, perchè avrei saltato più di k elementi. E' quindi sufficiente prendere il massimo fra i valori $DP[j]$, con $i - k - 1 \leq j < i$ e sommare il valore di $V[i]$. Ovviamente in presenza di 0 elementi il valore massimale è 0.

Attenzione però: il valore che cerchiamo non si trova in $DP[n]$ (che è la sottosequenza k -contigua massimale che termina nell'ultima posizione, anche se tale valore è negativo); non è nemmeno $\max(DP)$, perchè il vettore DP include sequenze contigue k -massimali che terminano anche molto prima di n . Dobbiamo quindi prendere il massimo incluso nel sottovettore $DP[n - k \dots n]$.

Il codice per risolvere il problema è il seguente:

```

int maxSumK(int[] V, int n, int k)
{
    int[] DP = new int[0...n]
    DP[0] = 0
    for  $i = 1$  to  $n$  do
    {
        DP[i] =  $-\infty$ 
        for  $j = \max(0, i - k)$  to  $i - 1$  do
        {
            DP[i] =  $\max(DP[i], DP[j] + V[i])$ 
        }
    }
    return  $\max(DP, n - k, n)$ 
}

```

La complessità è $O(nk)$, come la soluzione precedente. La complessità spaziale è però $O(n)$, un miglioramento rispetto alla soluzione precedente che aveva complessità spaziale $O(nk)$.